

Professional Certificate Programme in Agentic AI and Applications



Low-Code Tools Deep Dive



Revisit the Learning So Far

Let's take a quick look at the key ideas and concepts you have explored in your learning journey.

- How AI agents function and behave in real-world environments
- Practical use cases and applications of AI agents across industries
- Business ROI from adopting AI-driven agents in organisations
- Success stories and lessons learned from real-world implementations
- Workflow automation powered by AI agents across business processes
- Domain-specific brainstorming to uncover high-value use cases
- Case studies demonstrating measurable impact from AI agent deployment

Topics Covered

- Rise of Low-Code/ No-Code in AI Agents
- Tool Exploration
- Pros & Cons of Visual Pipelines
- Build a Multi-Tool Agent
- Debugging & Node Exploration
- Reflection & Wrap-Up

Agentic AI – Week 20

Low-Code Tools Deep Dive

Exploring the Rise of Low-Code/No-Code in AI Agent Workflows

Session Overview

Rise of low-code AI agents

- ✓ Understand the democratisation movement

Tool ecosystem

- ✓ Explore platforms such as Flowise, LangFlow and AutoGen Studio

Visual pipeline design

- ✓ Build & debug agent workflows

Hands-on practice

- ✓ Create a multi-tool agent together

Reflection & strategy

- ✓ Choose the right approach for your needs

Section 1

Rise of Low-Code/No-Code in AI Agents

From Code to Canvas



- LangChain scripts
- Python modules
- Command-line tools



- Drag-and-drop pipelines
- Interactive canvases
- Real-time previews



- Design logic visually
- Like drawing flowcharts
- Accessible to all



Traditional approach

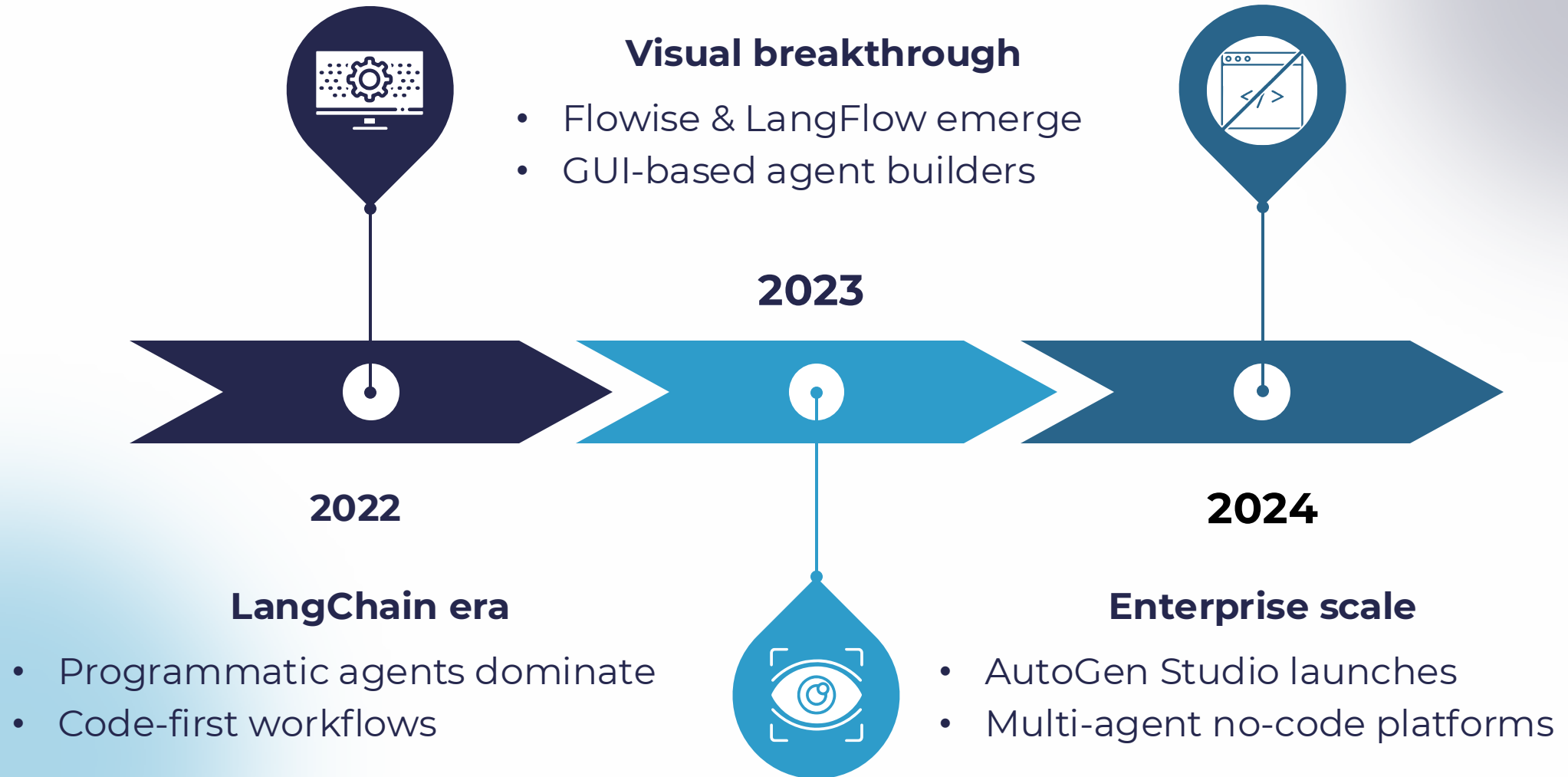


Visual revolution



New paradigm

Evolution Timeline



Why Low-Code Matters Now



Democratised access

Expand LLM
adoption across
industries for
non-technical roles



Speed to market

Accelerate
development
through rapid
prototype iterations



Enterprise integration

Connect seamlessly
with APIs and core
business tools

Key Principles of Low-Code Agent Design



Visual nodes

Each node represents a function or capability



Data flow links

Connections show how information moves



Debug panels

Real-time logs reveal runtime behaviour



State management

Memory & prompt blocks maintain agent context

Section 2

Tool Exploration

Flowise

Browser-Based Workflow Builder

Built on LangChain components

Leverages proven frameworks

Web-native experience

Requires no installation



Comprehensive support

Includes agents, chains and vector stores

Rich integration library

Connects easily to popular services

Demo 1 Build a Multi-Tool Agent in Flowise

LangFlow

Open-Source visual IDE

Provides complete transparency and community-driven development

Delivers a modern architecture designed for performance and flexibility

**React +
FastAPI
stack**

Local development

Runs directly on your machine for Python-based projects

Supports chain-development workflows with purpose-built tooling

**LangChain
native**

Demo 2 LangFlow Agent Creation

AutoGen Studio

Enterprise Multi-Agent Platform



Microsoft-powered

Built on Azure
infrastructure



Collaborative reasoning

Enables multiple
agents to work
together



Simulation environment

Allows complex
scenarios to be
tested safely



Enterprise scale

Designed for
production
workloads

Demo 3 Build an Agent Workflow in AutoGen Studio

Tool Comparison

Feature	Flowise	LangFlow	AutoGen Studio
Interface	Web UI	Web & Local	Cloud-based
Core focus	LLM Agents	LangChain development	Multi-agent simulation
Ease of setup	★ ★ ★ ★ ★	★ ★ ★ ★	★ ★ ★
Customization	Medium	High	High
Best for	Quick prototypes	Local dev teams	Enterprise scale
Pricing model	Free & paid tiers	Open source	Azure credits

Section 3

Pros & Cons of Visual Pipelines

Advantages



Faster prototyping

Build and iterate in minutes rather than hours

Cross-team collaboration

Align business, design and engineering around a shared workflow

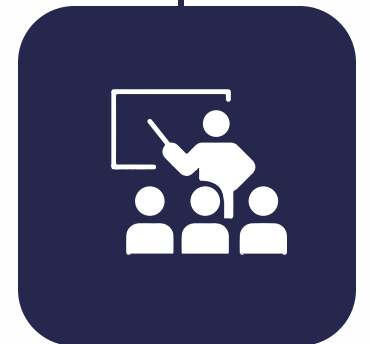


Visual debugging

Reveal exactly where issues occur in the flow

Teaching & demos

Explain complex concepts visually with clarity



Limitations



Version control challenges

Difficulty in tracking
Git workflow
changes



Customisation ceiling

Limitations when
logic exceeds
platform boundaries



Performance trade-offs

Overhead relative to
optimised code
execution



Vendor lock-in risk

Costs and delays
during migration
efforts

Discussion Prompt

"What's one advantage and one risk of using low-code AI tools in your organisation?"



Take 2 minutes to reflect individually, then share with a partner.

✓ **Consider your context**

Team size, technical capability, business goals

✓ **Think strategically**

Short-term gains v. long-term implications

✓ **Be honest**

Real challenges and genuine opportunities

Build a Multi-Tool Agent

Hands-on workshop: Create intelligent agents using visual workflows

Demo Setup

```
237     if access_type == WinregRegistryEntry.KEY_WOW64_32_BIT:
238         raise RuntimeError("Use either KEY_WOW64 or KEY_WOW32 with create_value()")
239
240     registry_key = None
241     wow64_flags = WOW64_MAP[access_type]
242     try:
243         key_hive_value = HIVES_MAP[key_hive]
244
245         if isinstance(value_type, RegistryKeyType):
246             value_type = TYPES_MAP[value_type]
247
248         registry_key = winreg.OpenKey(key_hive_value, key_path, 0, (wow64_flags | winreg.KEY_WRITE))
249         winreg.SetValueEx(registry_key, value_name, 0, value_type, key_value)
250         winreg.CloseKey(registry_key)
251         return True
252     except Error as e:
253         logger.error("Unable to write to registry path %s\\%s with LastError=%d [%s]",
254                     key_hive, key_path, e.winerror, e.strerror)
255         if registry_key is not None:
256             winreg.CloseKey(registry_key)
257         return False
258
259 def delete_value(key_hive, key_path, value_name, access_type=WinregRegistryEntry.KEY_WOW64_32_BIT):
260     """
261     Delete registry value
262     :param key_hive: Windows registry hive to edit, e.g. HKEY_CURRENT_USER
263     :param key_path: Path Windows registry key exists the hive, for example "SOFTWARE\\Classes\\txtfile"
264     """
```

✓ Platform

Flowise or LangFlow

✓ Goal

Agent with Calculator + Wikipedia Tools

✓ Process

Input → Reason → Tool Calls → Output

Workflow Structure



Input prompt block

Receives user questions and requests

Tool nodes

Executes actions such as Calculator + Wikipedia integration

Memory block

Maintains conversation context

Output node

Returns processed results

Step 1: Create a New Flow

Setup



Add LLM Node



Use GPT-4 mini or OpenAI API



Connect to Input Block

Foundation for your agent's reasoning engine

Step 2: Add Tool Nodes



Calculator Node
Drag from toolbox

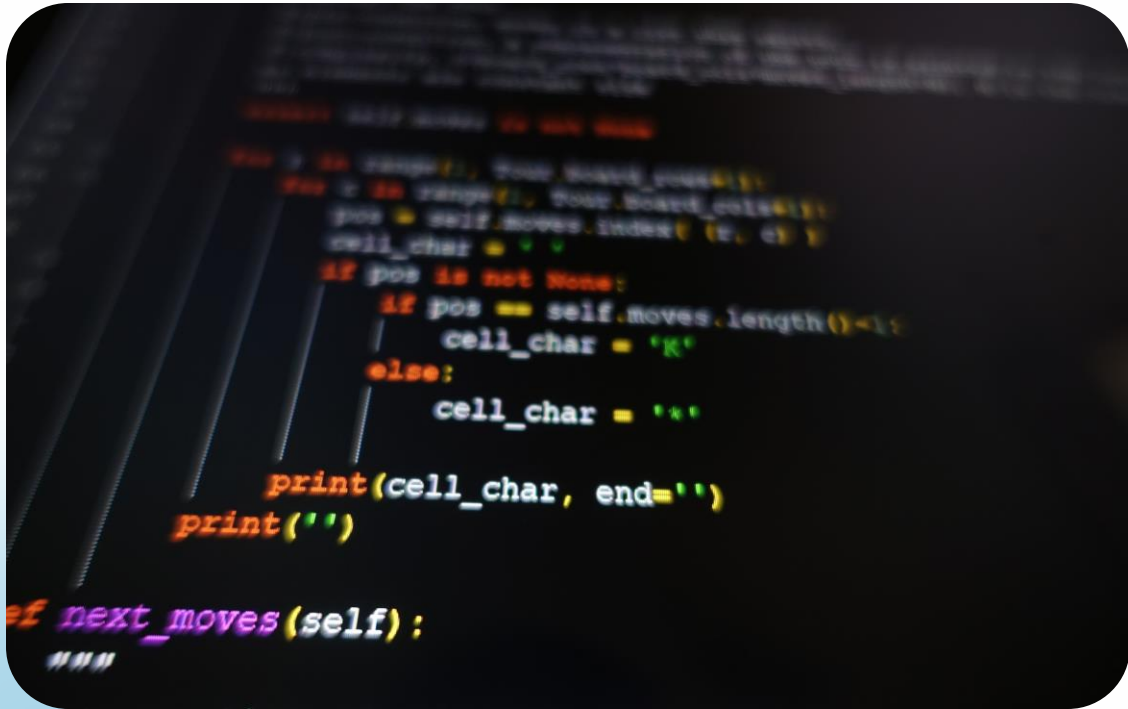


Wikipedia Node
Add to canvas



Link to LLM
Configure inputs/outputs

Step 3: Add Memory and Prompt Templates



```
for r in range(1, Four.Board.rows+1):
    for c in range(1, Four.Board.cols+1):
        pos = self.moves.index((r, c))
        cell_char = ''
        if pos is not None:
            if pos == self.moves.length()-1:
                cell_char = 'K'
            else:
                cell_char = 'x'
        print(cell_char, end='')
    print('')

def next_moves(self):
    """
```

Prompt node

Create template with placeholders:
{context}, {question}

Memory node

Add short-term memory for session context and conversation continuity

Step 4: Run and Observe

Execution monitor



Watch live logs



Track node execution order



View agent reasoning



Analyse tool usage patterns

Hands-On Challenge

“Add one custom tool (e.g., weather API or search node) and connect it to your flow.”



Extend your agent's capabilities with
external data sources

Debugging & Node Exploration

Tools and techniques for troubleshooting agent workflows

Debug View in Flowise



Token usage

Track API consumption and costs per execution

Prompt content

Inspect exact prompts sent to LLM

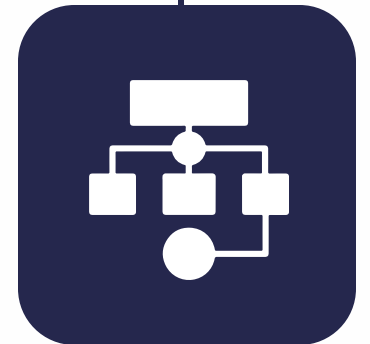


Latency metrics

Measure response times and bottlenecks

Visual trace

Follow path of tool calls through workflow



Modify Nodes Live



Edit prompts

Adjust instructions
mid-flow without
restarting



Memory depth

Configure context
window and history
length



Retry execution

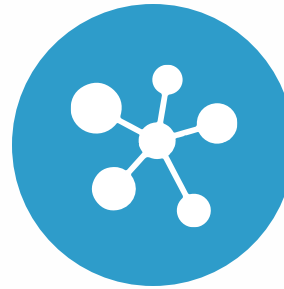
Test changes
immediately with
same inputs

Common Debugging Scenarios

Symptom
Authentication error

Fix
Verify environment variables
and credentials

API keys missing



Incorrect node linking

Symptom
Null output or broken chain

Fix
Check connection points and
data flow

Symptom
Invalid execution or template
errors

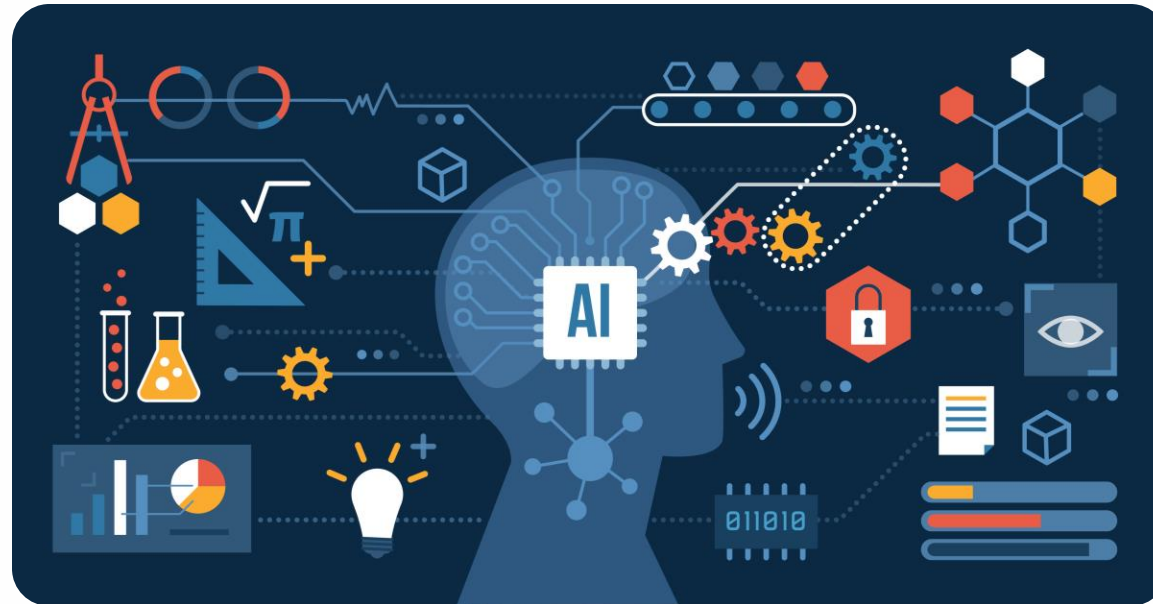
Fix
Ensure all placeholders are
defined

Missing prompt variables



Discussion

“What did you notice about agent reasoning steps in the debug logs?”



Share observations about decision-making patterns,
tool selection and context usage

Reflection & Wrap-Up

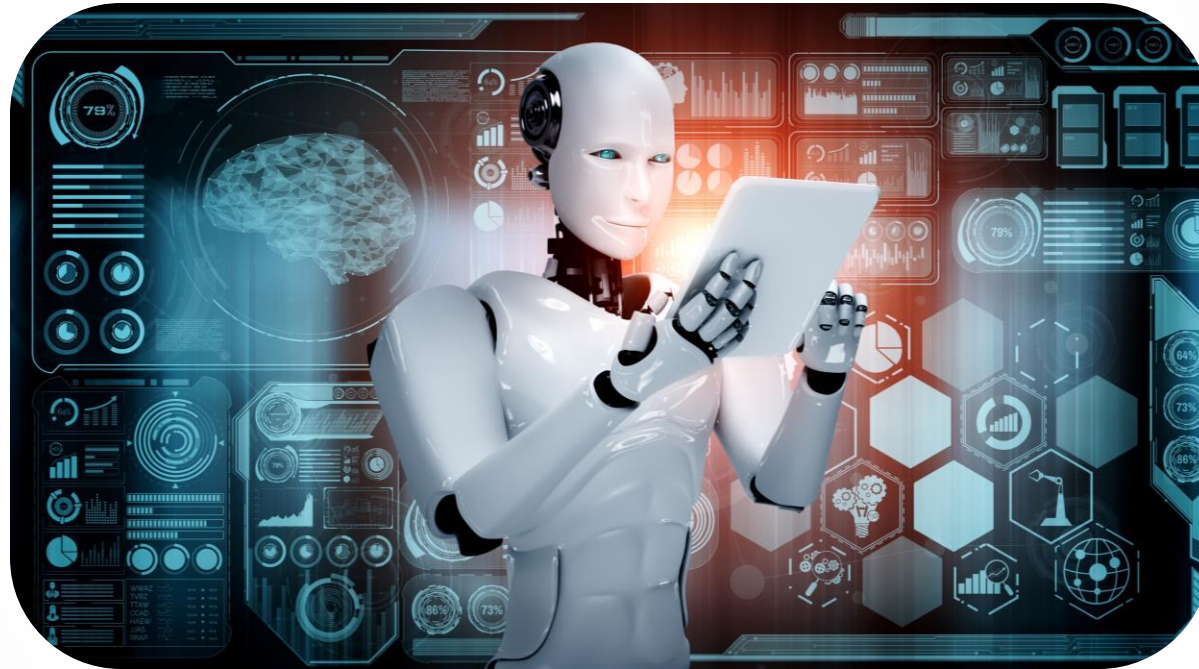
Connecting low-code tools with real-world AI development

Low-Code v. Code-First

Scenario	Recommended approach
Rapid prototyping & teaching	Low-Code
Complex custom logic	Code-First
Enterprise integration	Hybrid
Research & experimentation	Low-Code
Production MLOps pipeline	Code-First

Reflection Prompt

"When would you use low-code v. code-first agent building?"



Take 3 minutes to write a short journal entry reflecting on your learning

Key Takeaways

Low-Code AI = faster iteration, broader access

- ✓ Democratises AI development across technical skill levels

Visual flows bridge business + tech teams

- ✓ Create a shared language for collaboration and alignment

Debugging insight helps understand agent reasoning

- ✓ Provide transparency into decision-making processes

Balance speed v. control for each project

- ✓ Select the right tool based on project requirements

Next Week Preview



Week 21 Multi-Agent Collaboration & Workflow Orchestration

- Building agent teams with shared memory and goals

Resources



Flowise

<https://flowiseai.com>



LangFlow

<https://github.com/logspace-ai/langflow>



AutoGen

<https://autogen.microsoft.com>



LangChain Agents

<https://python.langchain.com/docs/modules/agents>

Q & A

Thank you